

Full Stack Development with MERN Project Documentation

1. Introduction

Project Title: [Customer-Registry]

Team Members: 1. NERELLA CHANDANA PRIYA

2. SAMPASALA AKHILA.

2. Project Overview:

- **Purpose:**

- **Define Structure:** Provides a structured design of all system components including Frontend, Backend, Database, API Layer, and Third-party Integrations
- **Clarify Component Interactions:** Maps out how data flows between UI, API, Services, and Database to avoid confusion during development
- **Select Optimal Technologies:** Justifies the choice of tech stack - React for UI, Node.js + Express for APIs, MongoDB for flexible data storage
- **Ensure Scalability & Performance:** Designs the system to handle increasing customer records, concurrent users, and faster response times using caching and load balancing
- **Improve Maintainability & Modularity:** Breaks the application into independent services so that changes in one module don't affect others
- **Strengthen Security & Compliance:** Defines authentication, role-based access control, data encryption, and audit logging to protect customer.
- **Enable Seamless Integration:** Specifies interfaces for integrating with Email services, SMS gateways, Payment systems, and Analytics tools
- **Guide Development Team:** Serves as a single reference blueprint for developers, testers, and DevOps to follow consistent standards
- **Support Deployment & Monitoring:** Outlines containerization, CI/CD pipeline, logging, and monitoring strategy for smooth delivery and operations
- **Future-Proof the System:** Creates a foundation that allows easy addition of new features like search, reporting, analytics, and AI without major rework

- **Futures:**

Customer Profile Creation: Allow users to create their customer profiles by providing essential details such as name, contact information, address, and any other relevant information.

Contact Management: Enable users to manage and organize their customer contacts effectively. Provide options to add, edit, and delete contact details, such as phone numbers, email addresses, and social media profiles.

Customizable Fields: Allow users to create custom fields to capture specific information unique to their business or industry. This feature ensures flexibility and adaptability to different customer data requirements.

User Registration: Allow customers, admins, and agents to register and create their accounts with unique login credentials.

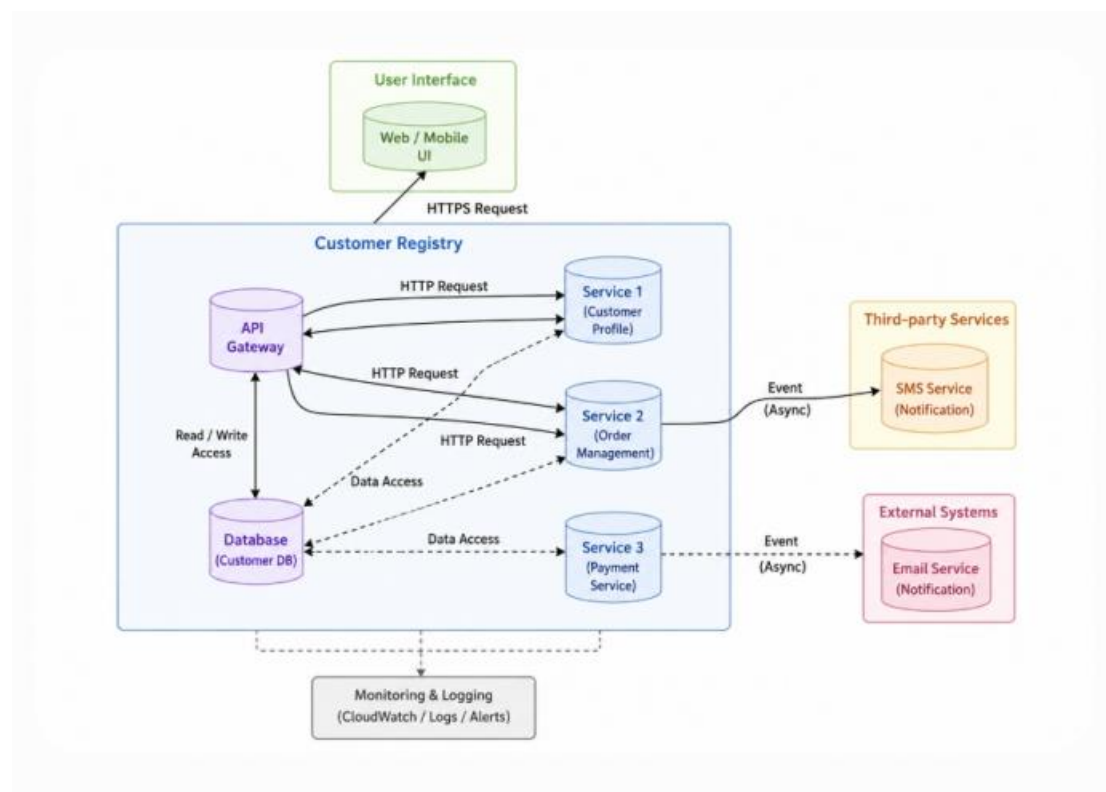
Admin Dashboard: Provide a dedicated dashboard for administrators to manage customer related data, user accounts, and system settings.

Agent Accounts: Allow agents to create profiles and access customer information for assistance and support purposes.

Customer Information Management: Enable administrators and agents to view and update customer information, including personal details, addresses, and communication preferences.

Communication Channels: Integrate communication channels like email or notifications to facilitate communication between customers, admins, and agents. **Communication History:** Maintain a record of all customer interactions, including emails, phone calls, meetings, and support tickets. This feature helps track communication history, ensuring a consistent and informed customer experience.

ARCHITECTURE:



A customer registry is a system that stores and manages information about customers or clients. It typically includes details such as names, contact information, addresses, purchase history, preferences, and other relevant data. The technical architecture of a

customer registry involves the components and technologies used to build and operate the system effectively.

Here are some key elements commonly found in a customer registry technical architecture:

Database: The database is the central storage component that holds all customer data. It can be a relational database management system (RDBMS) like MySQL or PostgreSQL, a NoSQL

database like MongoDB or Cassandra, or a combination of multiple databases depending on the requirements.

API (Application Programming Interface): The API serves as an interface for external systems to interact with the customer registry. It exposes a set of endpoints that allow authorized applications or services to perform operations such as creating, updating, retrieving, or deleting customer records. The API handles data validation, authentication, and authorization.

Services: Services are responsible for performing specific business logic and operations related to the customer registry. They can include services like customer registration,

authentication, customer profile management, data validation, and more. These services interact with the database and may communicate with each other as needed.

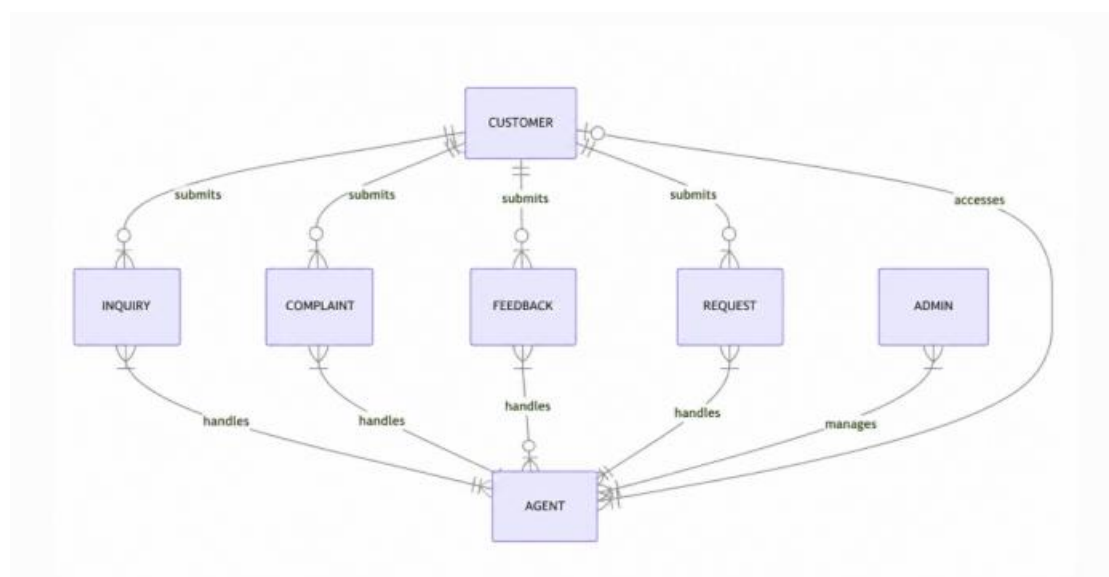
User Interface: The user interface component provides an interface for human users to interact with the customer registry system. It can be a web-based application, a mobile app, or even a command-line interface (CLI). The user interface communicates with the

API to perform actions such as searching for customers, updating information, or generating reports.

Integration with Third-party Services: A customer registry may integrate with various third party services to enhance its functionality. For example, it can integrate with email services to send notifications or marketing emails to customers. It can also integrate with SMS gateways for sending text messages. These integrations are typically based on event driven architecture, where certain actions or events trigger interactions with the third party services.

Security and Authentication: Security is a crucial aspect of a customer registry. It includes mechanisms for authentication and authorization to ensure that only authorized users or systems can access or modify customer data. Techniques like encryption, secure communication protocols (HTTPS), and user permissions play a vital role in securing the system.

Customer Care Registry ER-Diagram – Description:



A customer care registry ER-diagram represents the entities and relationships involved in a customer care system. It helps visualize how customers, inquiries, complaints, feedback, requests, agents, and administrators interact within the system. Here's a breakdown of the entities and their relationships:

CUSTOMER: Represents the individuals or entities that interact with the customer care system. They can submit inquiries, complaints, feedback, and requests.

INQUIRY, COMPLAINT, FEEDBACK, REQUEST: These entities represent different types of interactions or requests made by the customers. Each of these entities is associated with the CUSTOMER entity, indicating that a customer can submit multiple inquiries, complaints, feedback, or requests.

AGENT: Represents the agents or customer service representatives who handle the inquiries, complaints, feedback, and requests. Each of these entities is associated with the INQUIRY, COMPLAINT, FEEDBACK, or REQUEST entity, indicating that an agent can handle multiple customer interactions.

ADMIN: Represents the administrators or supervisors who manage the customer care system. The ADMIN entity is associated with the AGENT entity, indicating that an admin can manage multiple agents.

The relationships between the entities are represented using different notations. The "||-- o{" notation indicates a one-to-many relationship, where a customer can submit multiple inquiries, complaints, feedback, or requests. The "}|--|{" notation represents a many-to-many relationship, indicating that an agent can handle multiple inquiries, complaints, feedback, or requests, and each of these interactions can be handled by multiple agents. The "--o|" notation shows a one-to-one relationship, where an agent assists a single customer. The "}|--|{" notation also represents a many-to-many relationship between the ADMIN and AGENT entities, indicating that an admin can manage multiple agents, and each agent can be managed by multiple admins.

ROLES AND RESPONSIBILITY

Customer:

- Register and create an account in the customer registry app.
- Update and manage their profile information.
- Raise complaints or issues regarding products, services, or any other matters.
- View the status of their complaints and communicate with the assigned agent for updates and resolution.
- Provide feedback and ratings for the support received.

Admin:

- Manage the overall functioning of the customer registry app.
- Monitor and review customer complaints and issues raised.
- Assign complaints to the appropriate agents for resolution.
- Track the progress of complaints and ensure timely resolution.
- Have access to customer profiles and information for administrative purposes.
- Generate reports and analytics on customer complaints, resolution time, and agent performance.

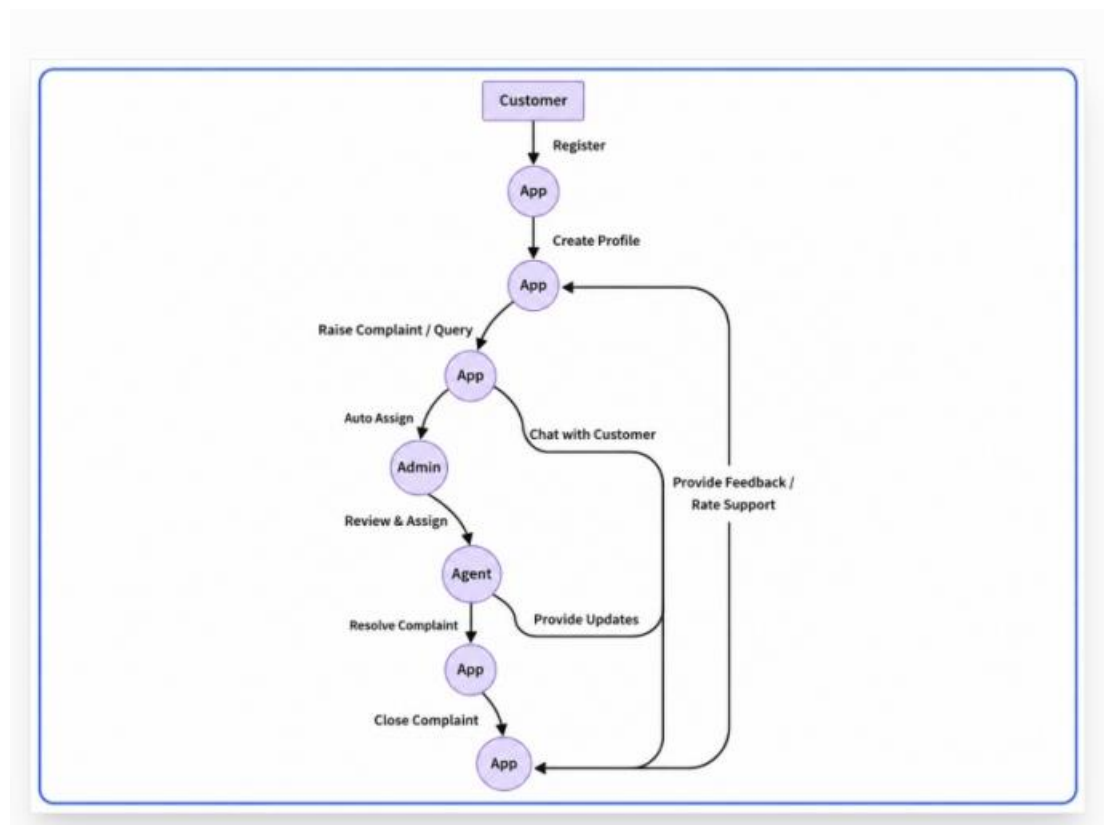
- Handle escalated complaints or issues that require special attention.
- Communicate with customers if necessary for clarification or additional information.

Agent:

- Receive assigned complaints from the admin.
- Communicate with customers to understand the issues and gather relevant details.
- Provide support and assistance to customers in resolving their complaints.
- Update the status and progress of complaints in the system.
- Communicate with customers through chat or other channels to provide updates and resolution.
- Escalate complex or unresolved complaints to higher levels if necessary.
- Maintain proper documentation of interactions and actions taken to resolve complaints.
- Collaborate with other agents or departments if required for complaint resolution.
- Provide feedback and suggestions to the admin for improving customer service.

It's important to note that these roles and responsibilities can be customized based on the specific requirements of your customer registry app. The level of access, authority, and communication channels may vary depending on the organizational structure and workflows within your company.

USER FLOW:



User Registration:

- User launches the customer registry app.
- User selects the option to register as a new customer.
- User enters their personal details such as name, email, phone number, and password. · User submits the registration form.
- The app validates the entered information and creates a new customer account. · The app displays a confirmation message and prompts the user to log in.

User Login:

- User launches the customer registry app.
- User selects the option to log in.
- User enters their login credentials (email and password).
- The app verifies the credentials and grants access to the customer dashboard.

Customer Dashboard:

- Upon successful login, the customer is presented with a personalized dashboard. · The dashboard displays relevant information, such as the user's profile, recent activities, and any notifications.
- The customer can navigate to different sections of the app, such as their profile, complaint management, or communication channels.

Raise a Complaint:

- From the customer dashboard, the user selects the option to raise a complaint. · The app presents a form or interface where the user can enter details about the complaint, including the nature of the issue and any relevant attachments.
- The user submits the complaint.
- The app validates the complaint and assigns it a unique complaint ID. · The app displays a confirmation message and provides the complaint ID for reference.

Admin Assignment:

- In the admin dashboard, the assigned administrator receives a notification about the newly raised complaint.
- The admin reviews the complaint details and assigns it to an appropriate agent for resolution.
- The app notifies the assigned agent about the new complaint assignment.

Agent Resolution:

- The assigned agent accesses the app and views the details of the assigned complaint. · The agent may initiate a chat or communication with the customer to gather additional information or clarify the issue.
- The agent works towards resolving the complaint, taking necessary actions, and updating the complaint status within the app.
- The agent communicates with the customer, providing updates and seeking feedback or confirmation of resolution.

- Once the complaint is resolved, the agent updates the status accordingly.

Customer Interaction:

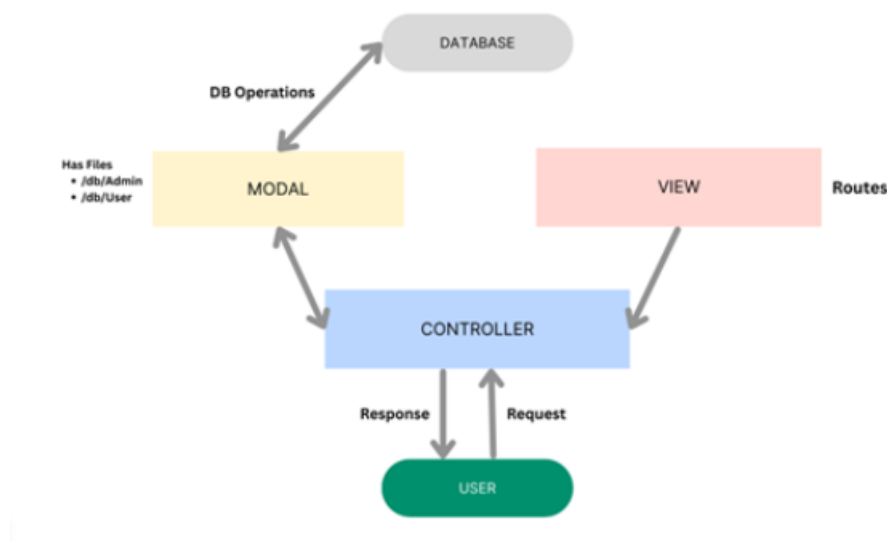
- The customer receives notifications or messages from the assigned agent regarding their complaint.
- The customer can access the app to view updates, respond to messages, or provide feedback on the resolution.
- The customer may engage in a chat conversation with the agent to discuss the complaint or seek further assistance.
-

Closure and Feedback:

- Once the complaint is marked as resolved by the agent, the customer is notified. · The customer can review the resolution and provide feedback or ratings for the support received.
- The app may prompt the customer to fill out a satisfaction survey or provide comments on their experience.
- The app records the feedback and updates the complaint status as closed.

This user flow provides a general outline of the typical steps and interactions involved in a customer registry app. However, it can be customized and expanded based on the specific requirements and features of your app.

MVC PATTERN:



The Customer Registry backend application follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability for this application.

Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using Mongoose, which provides a schema-based solution to model application data for MongoDB.

Controller Layer

The Controller layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer)

In the context of a backend REST API, the View is implemented as the routing layer, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Advantages of Using MVC in This Project

- **Separation of Concerns:** Each layer has a clearly defined responsibility, improving readability and maintainability.
- **Scalability:** New features can be added easily by creating new routes, controllers, and models.
- **Reusability:** Logic in controllers and models can be reused across multiple parts of the application.
- **Testing:** Each layer can be tested independently, especially the controllers and models.
- **Collaboration-Friendly:** Multiple developers can work simultaneously on different layers without conflict.
- **Frontend - React.js** Purpose: Provides the User Interface for customers and admins to interact with the system
- **Responsibilities:** Registration Form, Login, Customer Profile, Search, Dashboard Handles form validation and sends API requests to backend Manages state using React Hooks / Context API
- **Tech:** React.js, Axios for API calls, Tailwind CSS / Bootstrap for styling, React Router for navigation
- **Folder:** /client contains components, pages, services, and assets

Backend - Node.js + Express.js

- **Purpose:** Acts as the middle layer. Handles business logic, authentication, and API routing

Responsibilities:

- **API Layer:** Exposes RESTful endpoints for CRUD operations on customers
- **Services:** Customer Registration, Authentication, Profile Management, Data Validation
- **Middleware:** JWT Authentication, Error Handling, CORS, Request Logging
- **Tech:** Node.js, Express.js, JWT, bcrypt for password Hashing

- **Folder:** /server contains routes, controllers, models, middlewares, and config3.3
Database - MongoDB
Purpose: Central storage for all customer data
Responsibilities: Stores customer records: name, email, phone, address, preferences, purchaseHistory, createdAt
Supports flexible schema for future fields
Uses Mongoose ODM for schema validation and queries
Tech: MongoDB Atlas / Local MongoDB + Mongoose
Interaction: Backend connects to DB using Mongoose. Data is fetched and sent as JSON to Frontend

Database

- **MongoDB**
Purpose: Central storage for all customer data

Responsibilities:

- **Stores customer records:** name, email, phone, address, preferences, purchaseHistory, createdAt
Supports flexible schema for future fields
Uses Mongoose ODM for schema validation and queries
- **Tech:** MongoDB Atlas / Local MongoDB + Mongoose
- **Interaction:** Backend connects to DB using Mongoose. Data is fetched and sent as JSON to Frontend

4.Setup Instructions:

- **Prerequisites:** Before you start, make sure you have these installed on your system:
- **Node.js:** v18.x or above – Download
- **MongoDB:** Local MongoDB or MongoDB Atlas account
- **Git:** For cloning the repository
- **Code Editor:** VS Code recommended
- **Package Manager:** npm comes with Node.js

Installation :

Step 1: Clone the Repository

Bash

```
git clone https://github.com/your-username/customer-registry.git  
cd customer-registry
```

Step 2: Setup Backend

Bash

```
cd server  
npm install
```

Create .env file inside /server folder:

Java script

PORT=5000

MONGO_URI=mongodb+srv://<username>:<password>@cluster.mongodb.net/customer_registry

JWT_SECRET=your_jwt_secret_key

Install other dependencies:

Bash

```
npm install express mongoose cors dotenv bcryptjs jsonwebtoken
```

Step 3: Setup Frontend

Bash

```
cd ../client  
npm install
```

Install other dependencies:

Bash

```
npm install axios react-router-dom tailwindcss
```

Step 4: Configure Environment Variables

Create .env file inside /client if needed:

Javascript

REACT_APP_API_URL=http://localhost:5000/api

Running the Application Locally

Terminal 1 - Start Backend Server

Bash

cd server

npm start

Server will run on: <http://localhost:5000>

Terminal 2 - Start Frontend Server

Bash

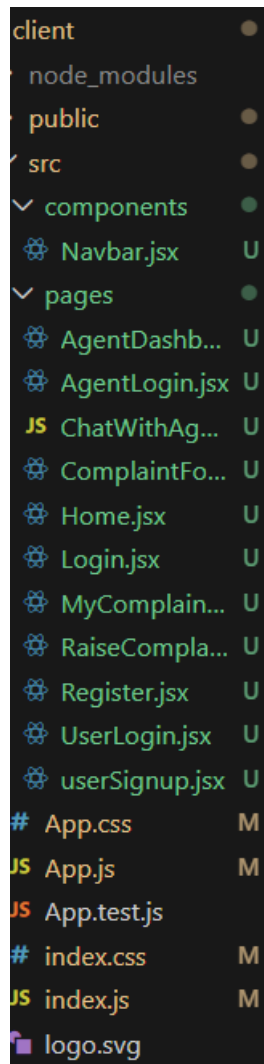
cd client

npm start

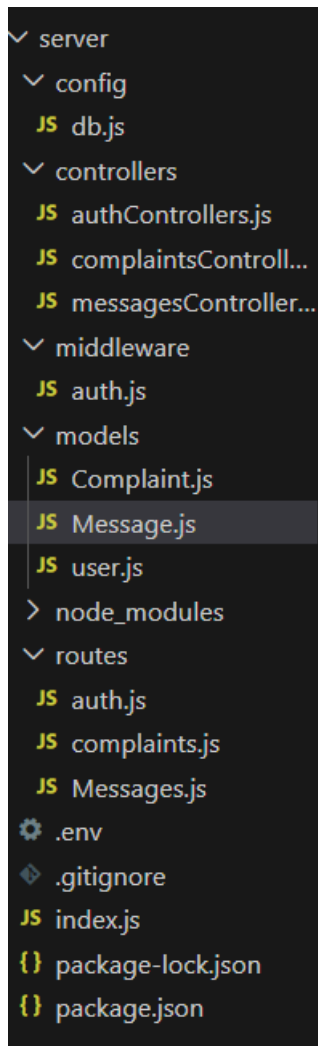
Frontend will run on: <http://localhost:3000>

5.Folder Structure

- **Client:**



- **Server:**



6. Running the Application:

- Provide commands to start the frontend and backend servers locally
 - **Frontend:** npm start in the client directory.
 - **Backend:** npm start in the server directory.

7. API Documentation:

- **Document all endpoints exposed by the backend**

Documenting all endpoints exposed by the backend is a critical part of any full-stack application. An API endpoint is basically a URL where your backend service can receive requests and send responses. In a MERN stack project like Customer Registry, the backend built with Node.js + Express.js exposes multiple RESTful endpoints. These endpoints act as the bridge between the React frontend and the MongoDB database.

Why we document all endpoints:

- **Clear Contract:**
- It defines exactly what operations the backend supports. Frontend developers know which URL to call, what method to use, and what data to send.

- **Avoids Confusion:** Without documentation, team members may duplicate APIs or use wrong parameters. Documentation gives one single source of truth.
 - **Faster Development & Testing:** QA and developers can test APIs independently using Postman/Swagger without waiting for the UI to be ready.
 - **Onboarding:** New developers can quickly understand the system by reading the API docs instead of going through the entire code.
 - **Maintenance & Scaling:** When you add new features like "Search Customer" or "Export Report", you just add new endpoints to the doc. This keeps the system organized.
 - **What to include while documenting endpoints:**
 - **Route/URL:** e.g. /api/customers/register
 - **HTTP Method:** GET, POST, PUT, DELETE
 - **Description:** What the endpoint does
 - **Request Parameters/Body:** What data is required to send
 - **Response:** Success response and Error response with status codes
 - **Authentication:** Whether JWT token is required or not
- For the Customer Registry project, all backend logic like Customer Registration, Login, Get Profile, Update, Delete are exposed as separate endpoints. Documenting them ensures the React frontend can call them consistently.
- **Include request methods, parameters, and example responses**
 - Request Methods - HTTP Verbs**
 - These define the action to be performed on the resource.
 - **GET** : Retrieve data. Ex: Get all customers or get customer by ID
 - **POST** : Create new data. Ex: Register a new customer
 - **PUT** : Update existing data. Ex: Update customer profile
 - **DELETE** : Remove data. Ex: Delete a customer
 - Using standard methods makes the API predictable and REST-compliant.

1. Parameters

Parameters are the data sent to the backend. There are 3 types:

- **Path Parameters:** Part of the URL. Ex: /customers/:id → id is the path parameter
- **Query Parameters:** Sent after ? in URL.
Ex: /customers?city=Chirala
- **Request Body:** JSON data sent in POST and PUT. Ex: {name, email, password} We must document what fields are required, data type, and validation rules.

2. Example Responses

This shows what the frontend will receive after calling the API.

We document both Success Response and Error Response with status codes.

- 200 OK / 201 Created → Success with data
- 400 Bad Request → Validation error
- 401 Unauthorized → Invalid or missing token

- 404 Not Found → Resource not found
- 500 Server Error → Backend issue

Example for Customer Registry API

Endpoint: POST /api/customers/register

Method: POST

Description: Register a new customer

Request Parameters - Body:

Json

```
{
  "name": "string - required",
  "email": "string - required, unique",
  "phone": "string - required",
  "password": "string - required, min 6 chars",
  "address": "string - optional"
}
```

Example Response - 201 Created:

Json

```
{
  "success": true,
  "message": "Customer registered successfully",
  "customer": {
    "_id": "64f1a2b3c4d5e6",
    "name": "Ravi Kumar",
    "email": "ravi@gmail.com"
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5In0.eyJ1IjoiRavi Kumar",
}
```

Example Error Response - 400 Bad Request:

```
{
```

```

    "success": false,
    "message": "Email already exists"
  }

```

8. Authentication:

- Login Flow Diagram The login flow explains step-by-step how a user gets authenticated and accesses protected routes.

javascript

[React Frontend] [Express Backend] [MongoDB]

```

|||
1. User enters Email & Password ||
|----- POST /login ----->||
|||
| 2. Find user by Email |
| |----- Query DB ----->|
| |<----- Return User Data -----|
|||
| 3. Compare password with bcrypt |
|||
| 4. If valid, Generate JWT Token |
|<----- Return Token + User Data ----| |
|||
5. Store Token in LocalStorage ||
|||
6. Send Token in Header for ||
Protected Requests ||
|----- GET /customers -----| |
|||
| 7. Verify Token with Middleware |
|||
|<----- Return Customer Data -----| |

```


Flow Explanation:

- **User Login:** User submits email + password from React Form
- **Validation:** Backend checks if email exists in MongoDBPassword Check: bcrypt.compare() verifies the hashed passwordToken
- **Issue:** If valid, backend creates JWT with {userId, role} and sends it back
- **Storage:** Frontend stores token in localStorage or sessionStorageProtected **Access:** For every future request, token is sent in Authorization header
- **Verification:** Backend middleware verifies token before allowing access

Sample Middleware Code for JWT Verification

This middleware runs before protected routes to check if the user is authenticated. File: server/middleware/authMiddleware.js

javascript

```
const jwt = require('jsonwebtoken');

const authMiddleware = (req, res, next) => {
  // 1. Get token from header
  const token = req.header('Authorization')?.split(' ')[1];

  // 2. Check if token exists
  if (!token) {
    return res.status(401).json({ success: false, message: 'No token, authorization denied' });
  }

  try {
    // 3. Verify token using secret key
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // 4. Attach user data to request object
    req.user = decoded.user;

    next(); // move to next route handler
  }
  catch (err) {
```

```

    res.status(401).json({ success: false, message: 'Token is not valid' });
  }
};

```

```

module.exports = authMiddleware;

```

How to use in Routes

```

const authMiddleware = require('./middleware/authMiddleware');

```

```

// Protected Route - Only logged-in users can access

```

```

router.get('/profile', authMiddleware, (req, res) => {
  res.json({ message: "Welcome to your profile", user: req.user });
});

```

```

// Admin Only Route

```

```

router.get('/customers', authMiddleware, adminMiddleware, getAllCustomers);

```

9. User Interface

Description: The User Interface of the Customer Registry - MERN Project is the part of the application that users directly interact with. It is developed using React.js + Tailwind CSS to provide a clean, responsive, and user-friendly experience across desktop, tablet, and mobile devices. The main goal of the UI is to make customer registration, login, and data management simple with minimal clicks.

9.1 Key Features of the UI

Responsive Design:

- The interface adjusts automatically for mobile, tablet, and desktop using Tailwind CSS grid and flex utilities.

User Authentication Pages :

- **Login Page:** For existing users to enter email & password
- **Register Page:** For new customers to create an account Form validation and error messages shown in real-time

Dashboard:

After login, users see a dashboard with options like: View Profile, Update Details, Logout

- **Admin Panel**

Admin can view all registered customers in a table format with Search, Edit, and Delete options.

- **Feedback & Alerts:** Success and error messages are displayed using toast notifications. Ex: "Registration Successful", "Invalid Credentials"

9.2 UI Components

Component	Purpose
Navbar	Navigation links: Home, Login, Register, Dashboard
RegistrationForm	Collect customer details: name, email, phone, address
LoginForm	Authenticate user and store JWT token
CustomerTable	Display all customers - Admin only
ProfileCard	Show logged-in user details
Footer	Project info and contact details

9.3 Design Principles Followed

- **Simplicity:** Minimal fields and clear buttons to avoid confusion
- **Consistency:** Same color scheme, fonts, and button styles across all pages
- **Accessibility:** Proper labels, contrast, and keyboard navigation
- **Performance:** Fast loading with component-based architecture in React

9.4 Technologies Used

- **React.js:** For building dynamic and reusable components
- **React Router DOM:** For page navigation without reload
- **Tailwind CSS:** For fast and responsive styling
- **Axios:** To connect UI with backend APIs

10. Testing

Description: Testing is the process of evaluating the Customer Registry - MERN Project to ensure it works correctly, is bug-free, and meets user requirements.

- The goal of testing is to find errors early, verify functionality, and ensure the system is reliable before deployment. In this project, we follow both Manual Testing and API Testing approaches.

Types of Testing Performed

Type	Purpose	Tools Used
1. Unit Testing	Test individual functions/components like registerUser, loginUser	Jest, React Testing Library
2. Integration Testing	Test connection between Frontend, Backend, and Database	Postman
3. API Testing	Verify all endpoints return correct response and status codes	Postman, Thunder Client
4. UI Testing	Check if forms, buttons, and pages work and are responsive	Manual testing on Chrome, Mobile
5. Validation Testing	Test form validations: required fields, email format, password length	Manual
6. Security Testing	Test JWT authentication, password hashing, and protected routes	Postman

Testing Process

- **Test Case Preparation**
Write test cases for each feature. Ex: "Register with valid email should return 201" API
- **Testing with Postman**
All backend endpoints like /register, /login, /customers/:id were tested with valid and invalid data.

JAVASCRIPT

Example: POST /api/customers/register

Input: {name, email, password}

Expected Output: 201 Created + JWT Token

- **Frontend Testing**

Checked if React forms submit data correctly to backend using Axios.
Verified error messages display when API fails.

- **Database Testing**
Verified that customer data is correctly saved, updated, and deleted in MongoDB Atlas.
- **Bug Fixing & Retesting**
Bugs found were fixed and the same test cases were run again to confirm.

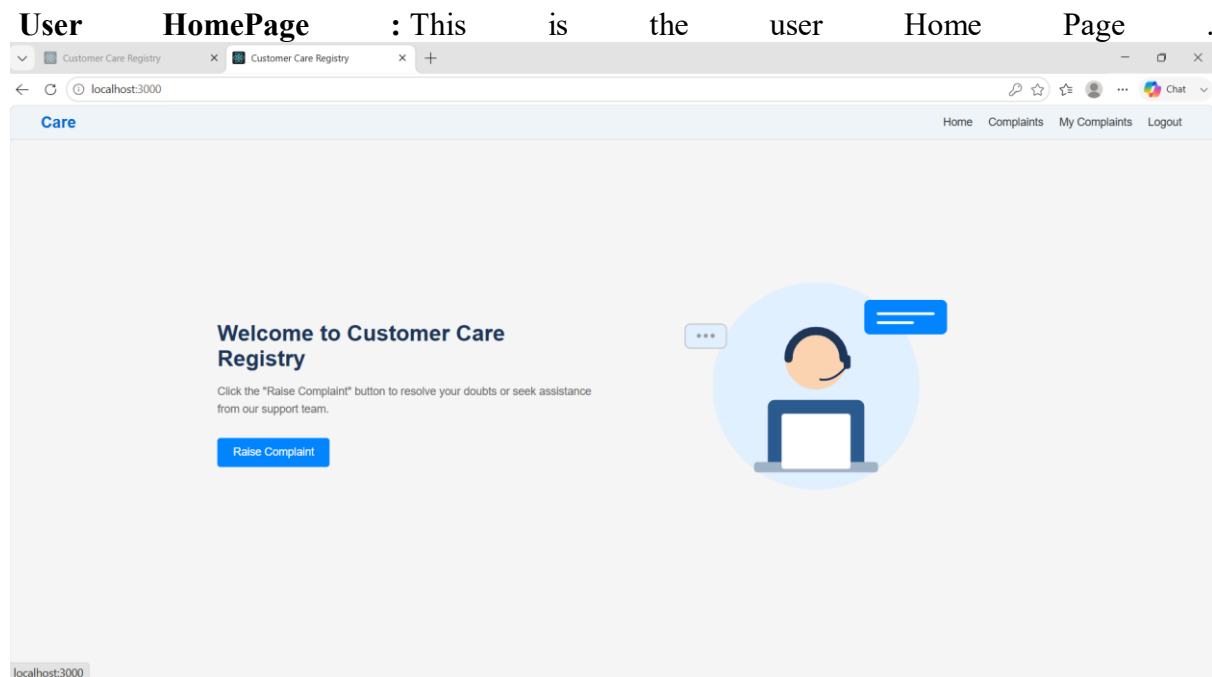
Test Results Summary

- All CRUD operations working as expected
- Authentication and authorization working with JWT
- Form validations preventing invalid data Entry
- UI is responsive on desktop and mobile
- No major bugs found in final testing phase

Importance of Testing

- **Quality Assurance:** Ensures the application works as intended
- **User Satisfaction:** Reduces crashes and bad user experience
- **Security:** Verifies that unauthorized users cannot access data
- **Reliability:** Makes sure the system is stable before deployment

11.Screenshots:



Login Page : This is the Login Page where users needs to login in to the application.

The screenshot shows a web browser window with two tabs labeled 'Customer Care Registry'. The address bar shows 'localhost:3000/login'. The page has a header with the word 'Care' on the left and navigation links 'Home', 'Complaints', 'My Complaints', and 'Logout' on the right. In the center, there is a white 'Login' form. The form contains an 'Email' field with the text 'chandana@2026', a 'Password' field with six asterisks, and a blue 'Login' button. Below the button is a link that says 'Don't have an account? Sign Up'.

Register Page : This is the Register Page where users needs to sign up to the application.

The screenshot shows a web browser window with two tabs labeled 'Customer Care Registry'. The address bar shows 'localhost:3000/register'. The page has a header with the word 'Care' on the left and navigation links 'Home', 'Complaints', 'My Complaints', and 'Logout' on the right. In the center, there is a white 'Sign Up' form. The form contains several fields: 'First Name' (chandana), 'Last Name' (priya), 'User Name' (chandana@2026), 'Email' (chandana@123), 'Password' (six asterisks), and a 'Type (admin, user, agent)' dropdown menu set to 'user'. There is a blue 'Sign Up' button at the bottom. Below the button is a link that says 'Already have an account? Login'.

Complaints Page : This is the complaints Page where users needs to Lodge a complaint

COMPLAINTS

Write Your Complaint

Name
Enter name

Phone
Enter phone

Email
Enter email

Complaint Details
Describe your complaint here...

Submit

My Complaints Page : This is the my complaints page where users can check there registered complaints

My Complaints

ID: 67f357eace032a9339246219
Complaint: Network error
Date: 7/4/2026
Status: pending

12. Known Issues

Description: Even after thorough testing, the Customer Registry - MERN Project may have some limitations and issues due to project scope, time constraints, and environment setup. Documenting these known issues helps developers and users understand what is not working perfectly yet and what can be improved in future versions.

List of Known Issues

S.No	Issue	Description	Impact
1	No Email Verification	During registration, email is not verified via OTP/link	Anyone can register with fake email
2	Password Reset Not Implemented	Users cannot reset forgotten password	User may get locked out
3	File Upload Not Supported	Customer profile image upload is not available	Limited profile customization
4	No Pagination in Admin Panel	If customers > 100, all data loads at once	Slow loading for large data
5	CORS Issue in Production	Sometimes frontend cannot connect to deployed backend	API calls fail after deployment
6	Token Expiry Handling	When JWT expires, user is logged out without warning	Bad UX
7	Basic UI Validation Only	Only frontend validation, no advanced validation	Invalid data may reach backend

Limitations

- **Single Role by Default:** Currently only Admin and Customer roles. More roles like Manager not added.
- **No Audit Logs:** System does not track who created/updated/deleted a customer.
- **Deployment:** Project is tested on localhost. Performance on cloud hosting not fully optimized.

Future Enhancements

To resolve these issues, the following can be added in Phase 2:

- Email OTP + Forgot Password
- Image Upload for Customer Profile
- Search, Filter, and Pagination for Customer List
- Role-baseddashboards
- Audit logs and Activity tracking

13. Future Enhancements

For Customer Registry - MERN ProjectThese are the improvements that can be added to make the system more useful for real businesses.

Key Enhancements

S.No	Feature	Short Description
1	Email OTP Verification	Verify customer email during signup to avoid fake accounts
2	Forgot Password	Reset password using email link
3	Profile Photo Upload	Customers can upload profile picture using Cloudinary
4	Search & Pagination	Admin can search customers by name/phone and view data page-wise
5	Export to Excel/PDF	Download customer list as report for records
6	Customer Dashboard	Show customer's own details, update history, and login activity
7	Email Notifications	Send welcome mail on registration and update alerts
8	Data Backup	Auto backup of MongoDB data weekly

